

Performance Analysis of QuickNode Versus Full Node

Ifesi Onubogu, Edward Stan, Ehsan Nikfar



Every Seconds Matters in the Modern World...

- Monitoring **Ethereum transactions** efficiently.
- Essential for **High-Frequency Trading**, where every millisecond counts, such as in high-frequency trading scenarios.
- **Front-running transactions**, a use case commonly associated with arbitrage opportunities in decentralized finance.
- Helps developers, researchers, and businesses
- The role of parallel processing in C++ in reducing delays and scaling performance.

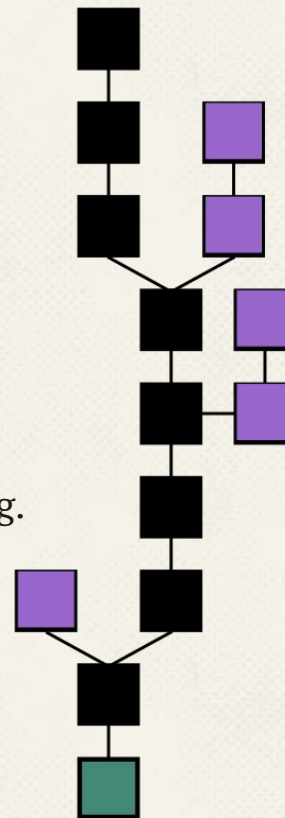


Crash Course on Blockchain

- Each block is a commit.
- The hash is the commit ID.
- The chain is the version history.
- Nodes are developers with a full repository clone.
- Consensus is resolving merge conflicts.
- Now you might understand what this [functional snippet] is doing.

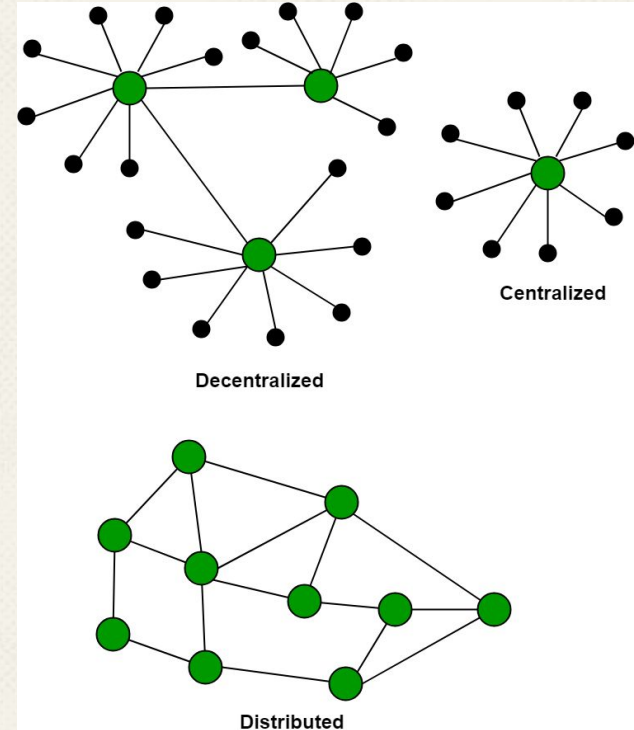
This was used to test in the live demo:

```
curl -X POST https://ifesifullnode.xyz \  
-H "Content-Type: application/json" \  
-u hello_you:42 \  
--data '{"jsonrpc":"2.0", "method":"eth_blockNumber", "id": 1}'
```



Introduction to Ethereum and Nodes

- A decentralized blockchain platform that powers smart contracts and decentralized applications (dApps).
- Unlike Bitcoin, Ethereum's blockchain is programmable, allowing for applications beyond simple peer-to-peer transactions.
- Ethereum Nodes: Play a crucial role by processing transactions and storing blockchain data to maintain the network.



What are Quick Nodes and Full Nodes and why they matter?

- **Full Nodes:**
 - **Self-hosted** and maintain a complete **copy of the blockchain**.
 - **Validate all transactions** and blocks, ensuring maximum security and reliability.
 - **Require significant resources:** high computational power, storage (~400GB), and synchronization time (~10 hours).
- **Quick Nodes:**
 - **Third-party providers** offering blockchain data services **via APIs**.
 - **Minimal resource requirements:** No local storage or full blockchain sync needed.
- Choosing between Quick Node and Full Node impacts **speed, scalability and reliability**

Motivation

- **Initial Focus:** Front-running in Ethereum, exploiting pending transactions for profit.
- **Optimization Goal:** Most tools are in Python; we aimed to improve them using C++.
- **Shift in Focus:** Pivoted to Full Node vs. Quick Node comparison due to ethical concerns.
- **Outcome:** Explored performance, scalability, and parallel processing in an ethical context.



Objectives and Design

- **Key performance metrics include:**
 - Time elapsed to process and match the transactions.
 - Responsiveness in detecting Uniswap Universal Router transactions
- **Full Node Monitoring:** Implemented with **full_node_monitor_mempool.cpp** to benchmark a Geth node.
- **Quick Node Monitoring:** Implemented with **quicknode_mempool_monitor.cpp** to benchmark Quick Node's API.
- **Performance Optimization:** Investigated the impact of **serial vs. parallel processing** on QuickNode transaction detection performance.

Design Methodology for quicknode_mempool_monitor.cpp

High Level Overview (SERIALIZED):

- The program **fetches** transaction hashes from the Ethereum Mainnet
- Query the Ethereum Mainnet to **retrieve** transaction details from **transaction hash**
- **Decode** transaction result & and relevant parameters (**TO**)
- **Filter** transactions directed **TO** Uniswap Universal Router
- Repeat until an arbitrarily specified amount of filtered transactions occur

What does the parallelized version do differently?

- Same as serial version, but add **decode** and **filter** operations to threadpool.

Implementation for quicknode_mempool_monitor.cpp

Fetching Transaction Hashes

- Set open handler to subscribe to pending transactions using QuickNode endpoint

```
ws_client.set_open_handler([&ws_client](websocketpp::connection_hdl hdl) {  
    std::string subscribe_message = R"({"jsonrpc":"2.0","id":1,"method":"eth_subscribe","params":["newPendingTransactions"]});"  
    ws_client.send(hdl, subscribe_message, websocketpp::frame::opcode::text);  
});
```

eth_subscribe RPC Method  QuickNode
Starts a subscription to a specific event. The [API credit](#) value for this method is .

Implementation for quicknode_mempool_monitor.cpp

Retrieve, Decode, and Filter Transaction Details

- Message Handler does all this
- Parallel version sends the body of this function straight into a 15-threaded threadpool, fueled by the modern library, **boost**.

body of message handler runs every time
websocket sends a new transaction hash

```
ws_client.set_message_handler([&ws_client, &transaction_hashes, &endpoint_url](websocketpp::connection_hdl, client::message_ptr  
msg) {  
    Json::Value root;  
    Json::CharReaderBuilder reader;  
    std::string errs;  
    std::istreamstream s(msg->get_payload()); // retrieve tx hash data  
    if (Json::parseFromStream(reader, s, &root, &errs)) {  
        std::string tx_hash = root["params"]["result"].asString(); // get transaction hash  
        // Retrieve and print transaction details  
        std::string tx_details = get_transaction_details(tx_hash, endpoint_url)  
        if (is_transaction_to_uniswap(tx_details)) { // FILTER transaction  
            std::cout << tx_hash << std::endl; // print out transaction hash  
        } // increment some global counter variable  
    }  
});
```

Implementation for quicknode_mempool_monitor.cpp

eth_getTransactionByHash RPC Method



Returns the information about a transaction from a transaction hash. The [API credit](#) value for this method is 20.

Get_transaction_details explored

- Retrieval and decoding of transaction data happens here

```
std::string get_transaction_details (const std::string& tx hash, std::string &endpoint url) {
    CURL* curl = curl_easy_init ();
    std::string result;
    if(curl) { // was curl initialized properly?
        std::string json request = R"({"jsonrpc":"2.0","method":"eth_getTransactionByHash","params":["] + tx hash + R"(", "id":1})";
        curl_easy_setopt (curl, CURLOPT URL, endpoint url.c_str()); // set target url for request
        curl_easy_setopt (curl, CURLOPT POST, 1L); // set request as POST
        curl_easy_setopt (curl, CURLOPT POSTFIELDS, json request.c_str()); // write request to POST field
        curl_easy_setopt (curl, CURLOPT WRITEFUNCTION, [](void* data, size_t size, size_t nmemb, std::string* writerData) -> size_t {
            if (writerData == nullptr) return 0;
            writerData->append((char*)data, size * nmemb);
            return size * nmemb;
        }); // callback function to handle response data
        curl_easy_setopt (curl, CURLOPT WRITEDATA, &result); // write response to result
        curl_easy_setopt (curl, CURLOPT HTTPHEADER, curl_slist_append (nullptr, "Content-Type: application/json" ));
        curl_easy_perform (curl); // run request
        curl_easy_cleanup (curl);
    }
    return result;
}
```

Implementation for quicknode_mempool_monitor.cpp

Great?

- Two API operations for 1 piece of data?
- One to get hash, another for transaction details of hash
- HTTPS requests *are expensive*
- 20 API credits for each API call...
 - Upgrade Tiers?

eth_getTransactionByHash RPC Method



Returns the information about a transaction from a transaction hash. The [API credit](#) value for this method is 20.

eth_subscribe RPC Method



Starts a subscription to a specific event. The [API credit](#) value for this method is 20.

System Design for full_node_monitor_mempool.cpp

1. Architecture Overview:

- The system interacts directly with a Geth full node via its **JSON-RPC API**. It performs the following core operations:
 - **Querying the Mempool** using **txpool_content** method to fetch details of all pending transactions from the Ethereum mempool.
 - **Parsing the Transactions**: extracts transaction details, such as the to address and the transaction hash.
 - **Filtering for Target Transactions**: compares each transaction's to address with the specified target address to identify matches.
 - **Logging and Benchmarking** : logs details of matching transactions and calculates the time taken to find a specified number of matches.

2. Key Components

Full node is the backbone of the system, and it operates as follows:

- **Data Synchronization:** Downloads and validates blocks to maintain a complete and up-to-date state of the Ethereum blockchain.
- **Mempool Management:** Maintains a pool of pending transactions awaiting inclusion in a block.
- **JSON-RPC API:** Exposes a programmable interface for querying blockchain data and interacting with the node.

2. Key Components

```
// Geth node URL
std::string ethereum_node_url = "http://localhost:8545";

// Uniswap Universal Router address
std::string uniswap_router_address = "0x3fc91a3afd70395cd496c647d5a6cc9d4b2b7fad";

// Helper function to make an HTTP POST request to query txpool_content
std::string get_txpool_content() {
    CURL* curl = curl_easy_init();
    std::string result;

    if (curl) {
        std::string json_request = R("{ \"jsonrpc\": \"2.0\", \"method\": \"txpool_content\", \"params\": [], \"id\": 1 }");

        curl_easy_setopt(curl, CURLOPT_URL, ethereum_node_url.c_str());
        curl_easy_setopt(curl, CURLOPT_POST, 1L);
        curl_easy_setopt(curl, CURLOPT_POSTFIELDS, json_request.c_str());
        curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, [](void* data, size_t size, size_t nmemb, std::string* writerData) -> size_t {
            if (writerData == nullptr) return 0;
            writerData->append((char*)data, size * nmemb);
            return size * nmemb;
        });
        curl_easy_setopt(curl, CURLOPT_WRITEDATA, &result);
        curl_easy_setopt(curl, CURLOPT_HTTPHEADER, curl_slist_append(nullptr, "Content-Type: application/json"));

        curl_easy_perform(curl);
        curl_easy_cleanup(curl);
    }

    return result;
}
```

2. Key Components

```
// Function to continuously check for transactions targeting the Uniswap Universal Router
bool find_and_print_uniswap_transaction() {
    std::string response = get_txpool_content();
    Json::Value root;
    Json::CharReaderBuilder reader;
    std::string errs;

    std::istream s(response);
    if (Json::parseFromStream(reader, s, &root, &errs)) {
        Json::Value pending = root["result"]["pending"];
        if (pending.isObject()) {
            // Iterate through all addresses in the pending transactions
            for (const auto& sender : pending.getMemberNames()) {
                const Json::Value& senderTxns = pending[sender];
                for (const auto& nonce : senderTxns.getMemberNames()) {
                    const Json::Value& tx = senderTxns[nonce];
                    if (tx.isMember("to") && tx["to"].asString() == uniswap_router_address) {
                        count += 1;

                        std::cout << "Transaction found to Uniswap Universal Router!" << std::endl;
                        std::cout << "Hash: " << tx["hash"].asString() << std::endl;

                        if (count == target) {
                            return true;
                        }
                    }
                }
            }
        } else {
            std::cout << "No pending transactions found in txpool." << std::endl;
        }
    } else {
        std::cerr << "Failed to parse JSON: " << errs << std::endl;
    }
    return false; // No matching transaction found
}
```


2. Key Components

```
while (true) {  
    if (find_and_print_uniswap_transaction()) {  
        break; // Stop looking once the target number of transactions is found  
    }  
    std::cout << "No transaction found, retrying..." << std::endl;  
    std::this_thread::sleep_for(std::chrono::seconds(2)); // Poll every 2 seconds  
}
```

2. Key Components

```
[Unit]
```

```
Description=Geth Full Node  
After=network-online.target  
Wants=network-online.target
```

```
[Service]
```

```
WorkingDirectory=/home/ubuntu  
User=ubuntu  
ExecStart=/usr/bin/geth --http --http.addr "0.0.0.0" --http.port "8545"  
--http.corsdomain "*" --http.api personal,eth,net,web3,debug,txpool,admin  
--authrpc.jwtsecret /tmp/jwtsecret --ws --ws.port 8546 --ws.api  
eth,net,web3,txpool,debug --ws.origins="*" --metrics --maxpeers 150  
Restart=always  
RestartSec=5s
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Demo Time!



Recorded demo

The screenshot displays the Etherscan.io website interface. At the top, the browser address bar shows 'etherscan.io'. The website header includes the Etherscan logo, navigation links for 'Home', 'Blockchain', and 'Tokens', and a user profile section for 'fesi Onubogu'. Below the header, the main content area is titled 'The Ethereum Blockchain Explorer'. It features a search bar with the text 'All Filters' and a search button. A sponsored banner for 'MetaWin' is visible, stating 'More than \$767,000 RAISED! Enter NOW & SECURE your chance to WIN!'. To the right, there is an advertisement for 'MoonPay' with the text 'A WHOLE WORLD OF CRYPTO, IN ONE SIMPLE ACCOUNT'. The main data section is divided into several panels. The 'ETHER PRICE' panel shows '\$3,861.15 @ 0.039408 BTC (+0.04%)'. The 'MARKET CAP' panel shows '\$465,044,851,258.00'. The 'TRANSACTIONS' panel shows '2,606.82 M (14.9 TPS)'. The 'MED GAS PRICE' panel shows '18.661 Gwei (\$1.51)'. The 'TRANSACTION HISTORY IN 14 DAYS' panel shows a line graph with data points for Nov 21, Nov 28, and Dec 5. Below these panels, there are two sections: 'Latest Blocks' and 'Latest Transactions'. The 'Latest Blocks' section shows two blocks: '21340364' (7 secs ago) and '21340363' (19 secs ago), both mined by 'Miner beaverbuild' and 'Miner Titan Builder' respectively. The 'Latest Transactions' section shows two transactions: one from '0xc6312b97be...' to '0x95222290...5CC4BAfe5' (7 secs ago) and another from '0x668e815d22...' to '0xaAaaAaa...8Df6CDC87'. A cookie notice is visible at the bottom of the page.

ETH Price: \$3,861.15 (+0.04%) Gas: 18.661 Gwei

Etherscan Home Blockchain Tokens Developers More Sign In

The Ethereum Blockchain Explorer

All Filters Search by Address / Txn Hash / Block / Token / Domain Name

Sponsored: MetaWin: More than \$767,000 RAISED! Enter NOW & SECURE your chance to WIN!

Ad MoonPay A WHOLE WORLD OF CRYPTO, IN ONE SIMPLE ACCOUNT

ETHER PRICE
\$3,861.15 @ 0.039408 BTC (+0.04%)

MARKET CAP
\$465,044,851,258.00

TRANSACTIONS
2,606.82 M (14.9 TPS)

MED GAS PRICE
18.661 Gwei (\$1.51)

TRANSACTION HISTORY IN 14 DAYS
1 360k
1 200k
Nov 21 Nov 28 Dec 5

Latest Blocks Customize

21340364 7 secs ago Miner beaverbuild 196 txns in 12 secs 0.05172 Eth

21340363 19 secs ago Miner Titan Builder

Latest Transactions Customize

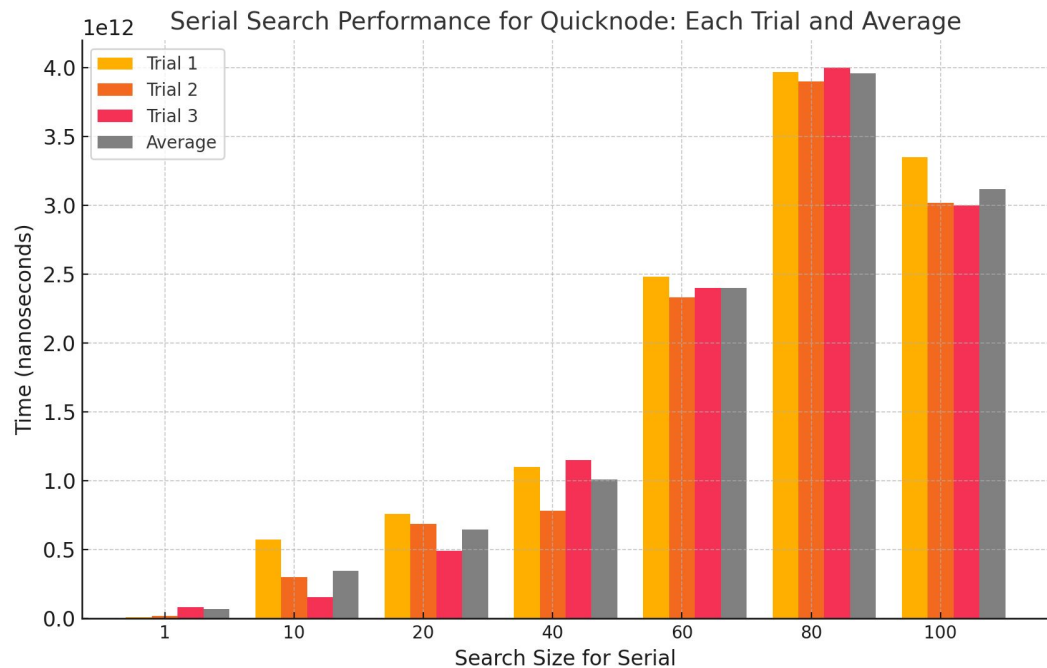
0xc6312b97be... 7 secs ago From 0x95222290...5CC4BAfe5 To 0x2bf916f8...252aE4087 0.04086 Eth

0x668e815d22... From 0xaAaaAaa...8Df6CDC87 0.13194 Eth

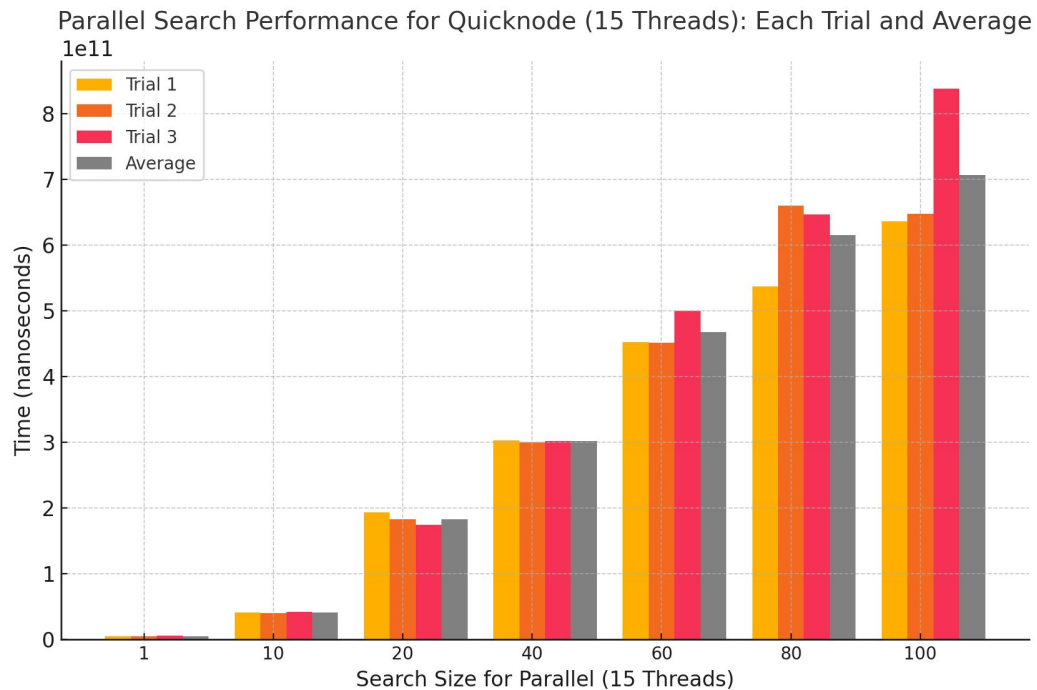
This website uses cookies to improve your experience. By continuing to use this website, you agree to its Terms and Privacy Policy. Got It!

**Is QuickNode actually
quick?**

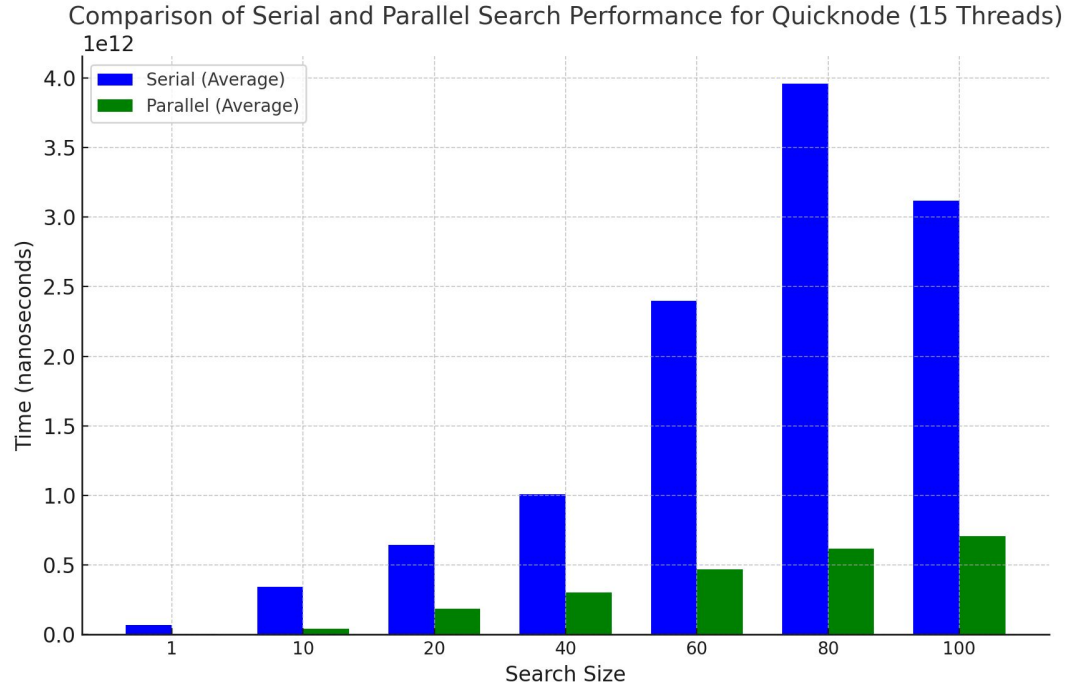
Serial Measurements For Quick Node



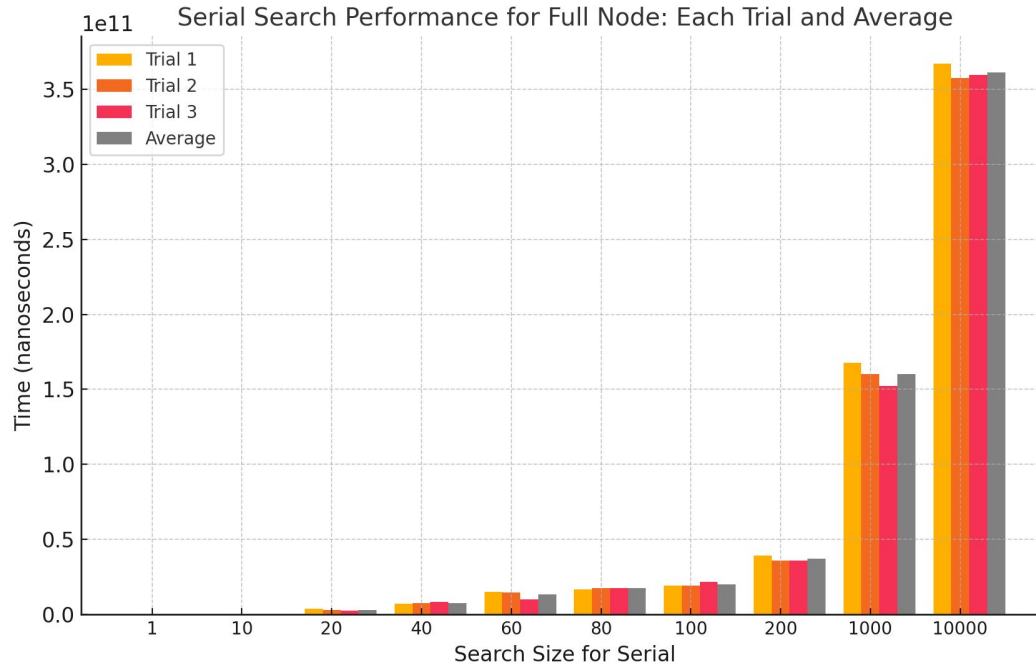
Parallel Measurements For Quick Node



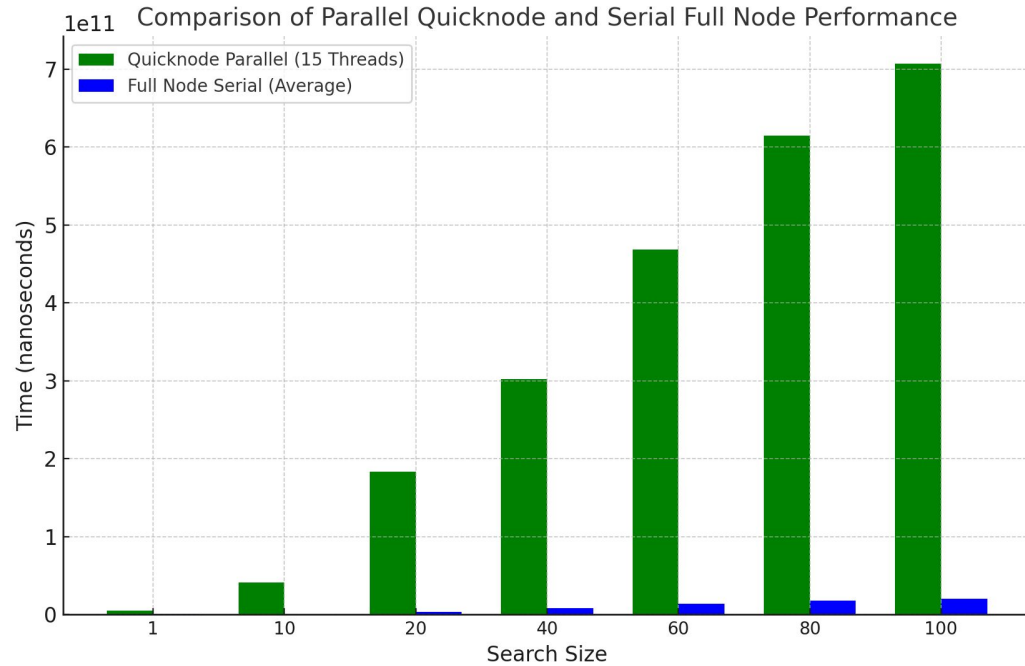
Serial vs Parallel for Quick Node



Measurements For Full Node

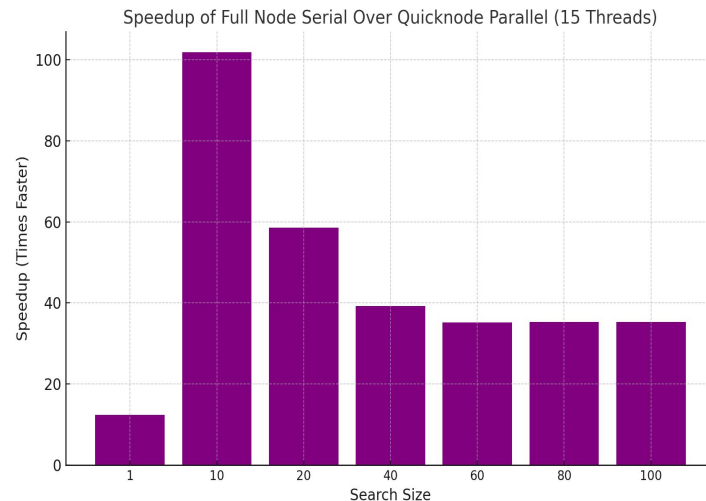


Full Node vs Quick Node



Measurements Discussion

- On average, the Full Node Serial is approximately 45.4 times faster than QuickNode Parallel (15 Threads).
- The primary bottleneck for QuickNode in serial mode is the **sequential handling of HTTP POST request that uses JSON-RPC for the payload.**



The bottleneck for QuickNode is the HTTP request

```
typedef websocketpp::client<websocketpp::config::asio_client> client;

std::string ethereum_node_url = "ws://localhost:8546"; // Geth WebSocket URL
std::string filter_address; // Address to monitor

// Helper function to fetch transaction details
✓ std::string get_transaction_details(const std::string& tx_hash) {
    CURL* curl = curl_easy_init();
    std::string result;

    if (curl) {
        std::string json_request = R"({"jsonrpc":"2.0","method":"eth_getTransactionByHash","params":[""] + tx_hash + R"(","id":1})";

        curl_easy_setopt(curl, CURLOPT_URL, "http://localhost:8545");
        curl_easy_setopt(curl, CURLOPT_POST, 1L);
        curl_easy_setopt(curl, CURLOPT_POSTFIELDS, json_request.c_str());
        curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, [](void* data, size_t size, size_t nmemb, std::string* writerData) -> size_t {
            if (writerData == nullptr) return 0;
            writerData->append((char*)data, size * nmemb);
            return size * nmemb;
        });
        curl_easy_setopt(curl, CURLOPT_WRITEDATA, &result);
        curl_easy_setopt(curl, CURLOPT_HTTPHEADER, curl_slist_append(nullptr, "Content-Type: application/json"));

        curl_easy_perform(curl);
        curl_easy_cleanup(curl);
    }
    return result;
}
```

Costs

- TL,DR

Running a full node is
expensive ... but how much
more expensive?

Costs

- Quicknode costs \$39.
- I come with receipts

Total: **\$39.00** USD*

Paid with: **USD**

Transaction ID:

ch_3QR34sJAMcsjcvZy0GUPMlcq

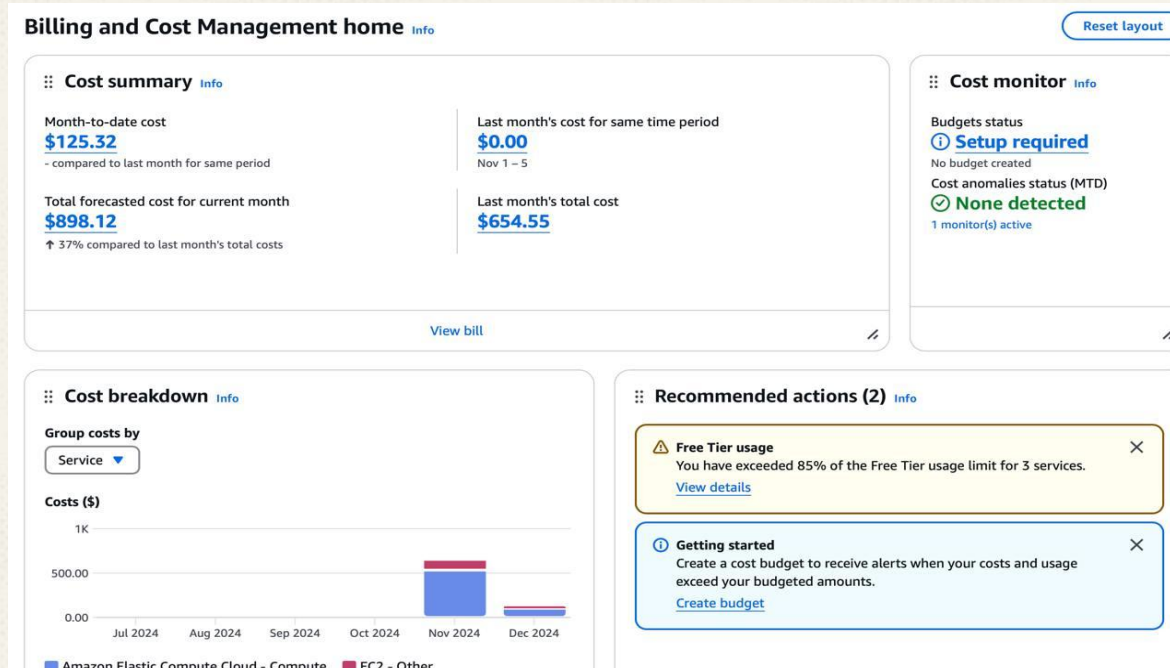
Date: **December 01, 2024**

For service through: **January 01, 2025**

[View Invoice](#)

Costs

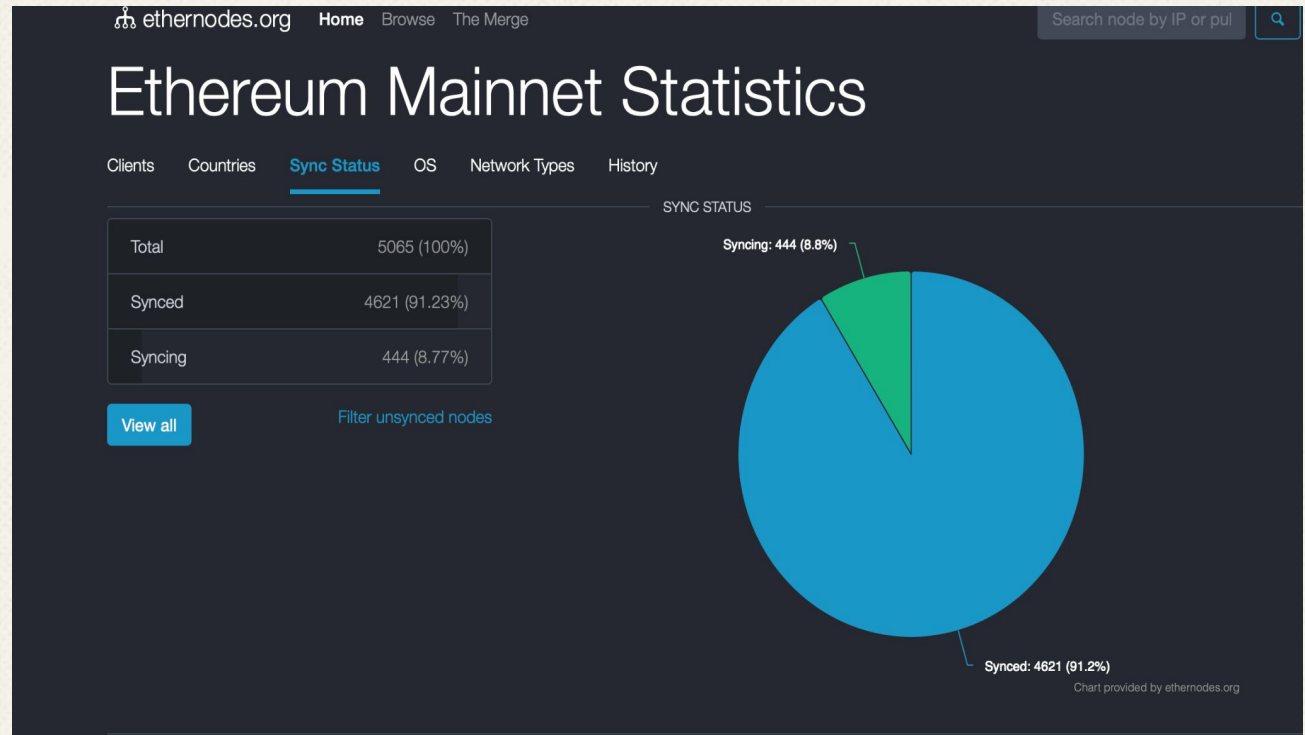
- The full node is this much more expensive



[illegible]

- **Full node provides unmatched performance but is a single point of failure**
- **The failure in question:**

[illegible][illegible]



What matters is the use case. This is what the ethereum mainnet looks like – decide what you want to do with it and pick your tools accordingly.

Insights

- **QuickNode's serial implementation is significantly slower than the full node**
- **Parallelizing QuickNode greatly improves performance but still lags behind the full node**
- **Full node provides unmatched performance but is a single point of failure**
- **QuickNode is cost-effective and resilient but sacrifices performance**
- **Use cases should dictate the choice of infrastructure**

Acknowledgements

- **Blockchains Explained**
 - <https://www.youtube.com/watch?v=Jc0NwgqCtik&feature=youtu.be>
- **Inspiration for the project, and good guide on front-running.**
 - <https://medium.com/@ivanbogatyev/front-running-bancor-in-150-lines-of-python-with-ethereum-api-d5e2bfd0d798>
- **Guide on front-running**
 - <https://www.youtube.com/watch?v=uElOqz-Htos&feature=youtu.be>
- **Libraries Used**
 - boost, websocketpp, openssl@3, curl, jsoncpp
 - The C++ Standard Library

Individual Contribution

- Ifesi Onubogu (io2249@columbia.edu)
 - Researched and implemented full node mempool monitoring , documented the project.
- Edward Stan (aes2385@columbia.edu)
 - Researched and implemented quicknode mempool monitoring
- Ehsan Nikfar (en2572@columbia.edu)
 - Implemented the initial Front-running program in C++

Future Directions and More

- Implementing the parallel version for Full Node
- Online Tutorials available to learn more about it, such as **Crypto Wizards** on YouTube.
- Implementing Ethereum trading front-runs on the Bancor exchange in Python

THANKS!



“it’s the best i can do - i’m having an orange party atm”

Any questions?